COMS W4156 Advanced Software Engineering (ASE)

September 14, 2023

Attendance

We will start taking attendance next Tuesday using zoom's 'usage reports'

These show each attendee's zoom name, join time, leave time, and duration for everyone who attended that zoom session

Make sure your zoom name matches your courseworks name, or you will not get credit for attending. If there is some reason you cannot make them match you need to explain (by posting a private message to the teaching staff, not just me, in edstem)

Make sure you stay for the whole session unless you tell us (by posting a private message to the teaching staff, not just me, in edstem) why you had to arrive late and/or leave early

If you have a bad connection and you manually disconnect and reconnect, or zoom disconnects you and you reconnect, we can put the two (or more) together as long as it's the same zoom name

"Looking for Group"

To Be Filled In On The Fly

After I open the breakout rooms

room 1 - C++

room 2 - Java

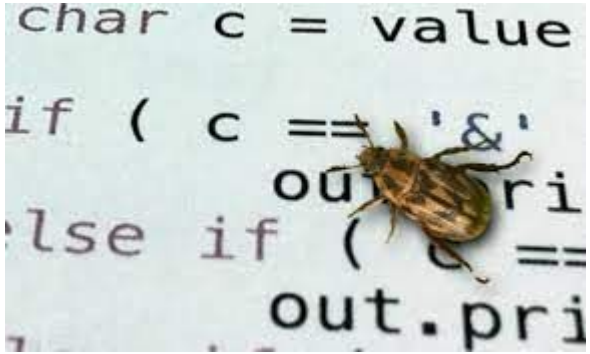
room 3 - Javascript

This is only for students still looking for a team and teams who need more members.

The other rooms are for fully formed teams to talk to each other while we're trying to form other teams, if they want, this is entirely optional. Pick one that isn't occupied or just hang around in the main room.

Agenda

1. Looking for Group
2. Finding Bugs by examining the code
3. deferred until a later date: Finding Bugs by executing the code

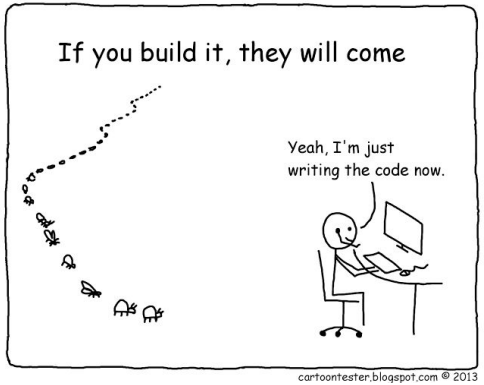


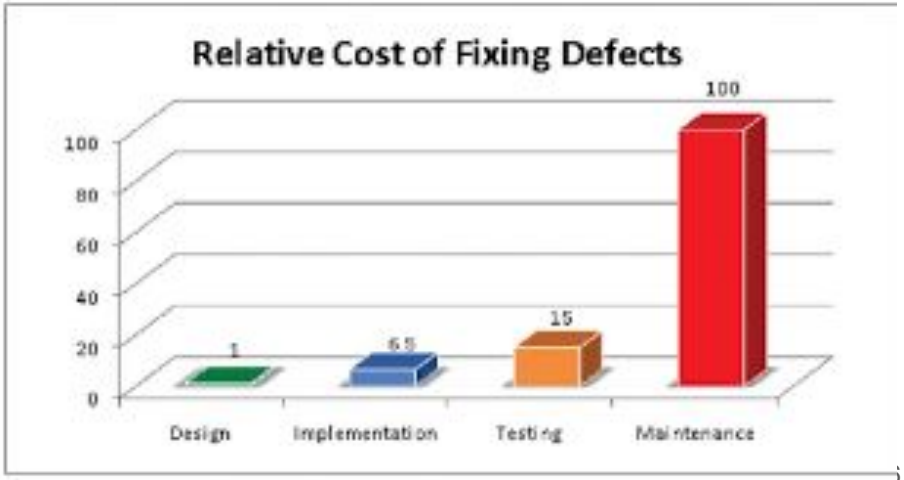
Bugs Are Everywhere

[Software has bugs. This is normal]

Buggy code does not matter much if it never leaves the developer's machine

Buggy code committed to shared repositories will increase time and effort for *other* software engineers, but unlikely to be humongously expensive if bugs never make it to production



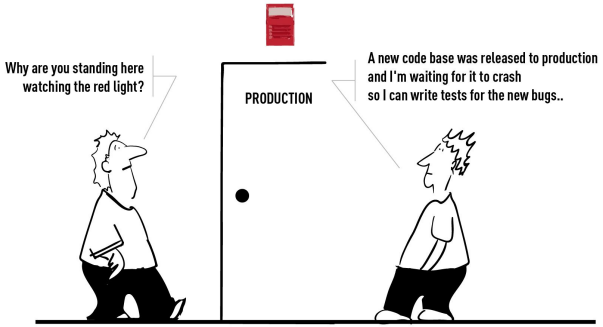


Preventing Buggy Code from being Released to Production

Human Code Review is labor-intensive (expensive!) and limited by human memory and patience

AI Code Review is limited by context length

What can be Automated *at Scale*?



Automatically Finding Bugs *at Scale*

Examine the code: Static Analysis

Execute the code: Dynamic Analysis

~~Formal Methods~~

Static Analysis

Any program analysis technique that does not actually execute the code is 'static'

Static analyzers do not need to know anything about what your code is supposed to do, they look for 'generic' problems that occur in many programs

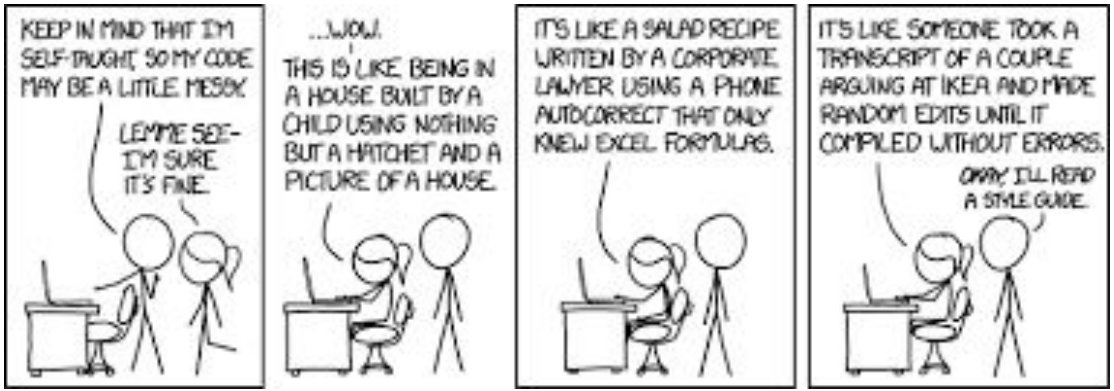
Includes style checkers: non-compliance with coding standards does not necessarily mean there's a bug, but does make it more likely for bugs to occur as multiple developers (mis-)understand and modify the code

But the term static analysis or static analyzer more commonly refers to 'code smell' detectors and bug finders, typically combined, like Infer and SonarQube

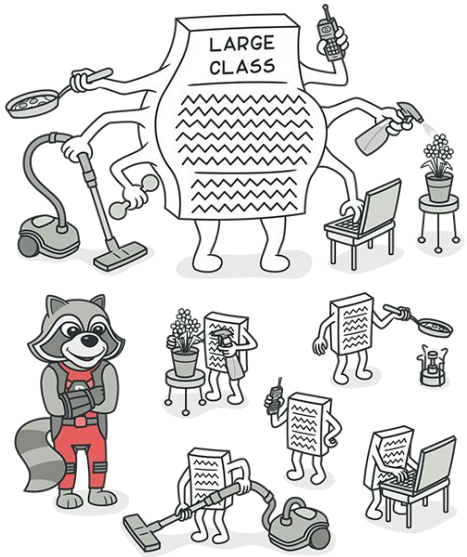
Code Smells

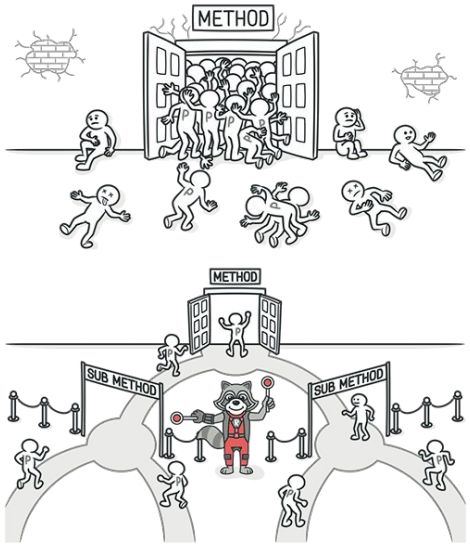
A '[code smell](#)' is a surface indication that usually corresponds to a deeper problem in the code

Like poor coding style, code smells do not necessarily indicate a bug, yet, but tend to make changes more difficult and more likely to lead to bugs as multiple developers (or your future self) modify the code

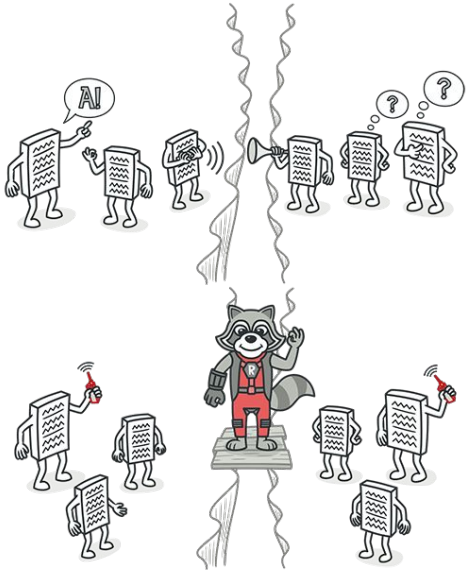


Example Code Smells: Bloaters





Example Code Smells: Couplers



more code smell examples [here]

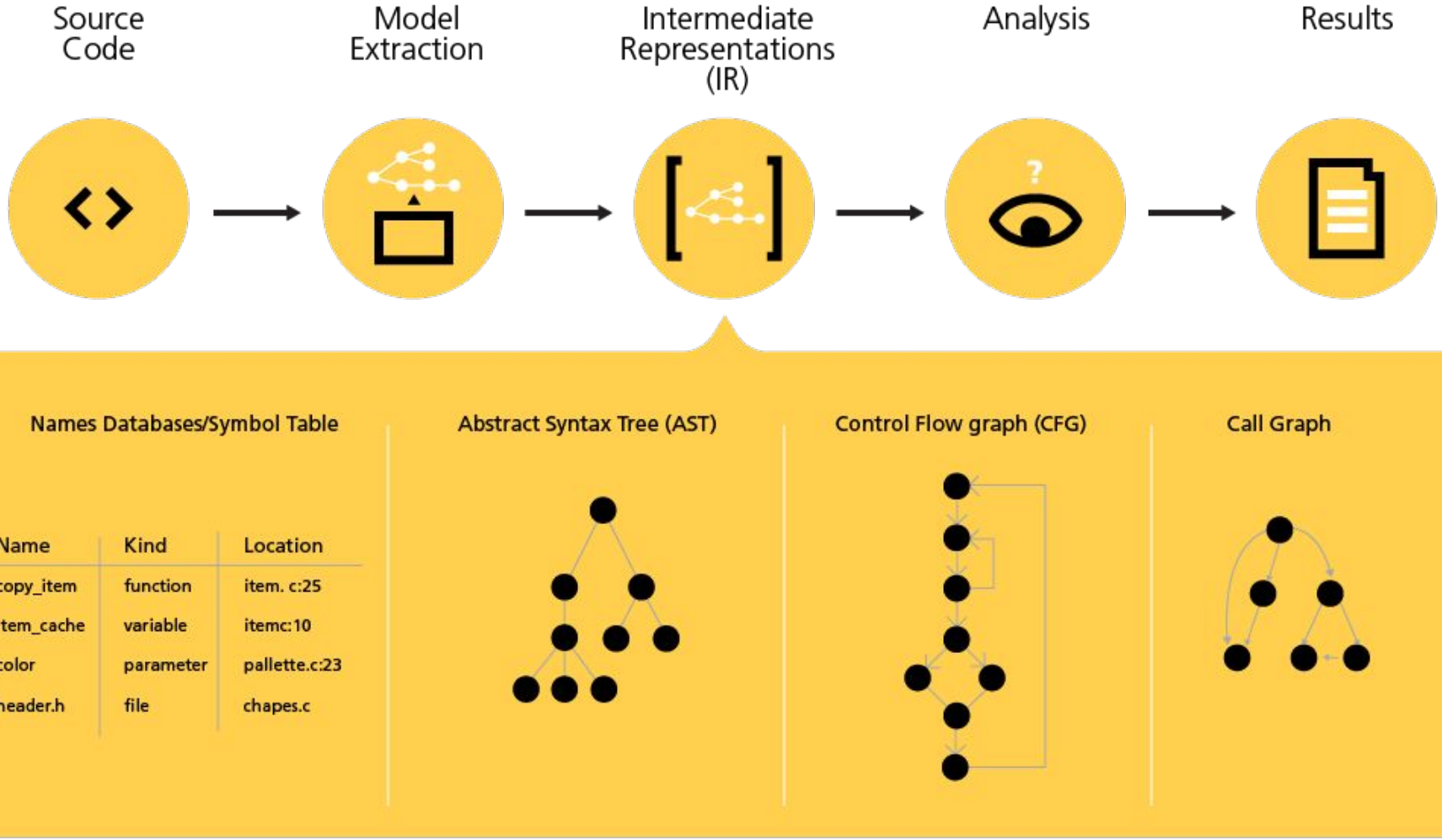
Bug Finders

Style checkers usually work by scanning the code as text

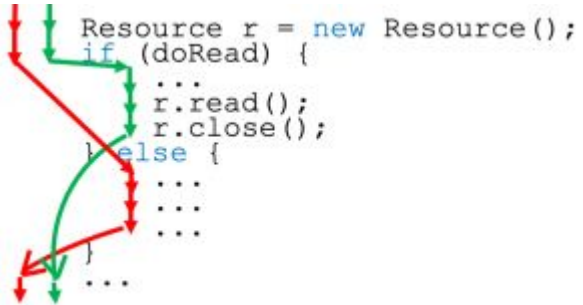
Code smell detectors and bug finders instead look for 'patterns' in an internal representation of source code (sometimes binary/bytecode):

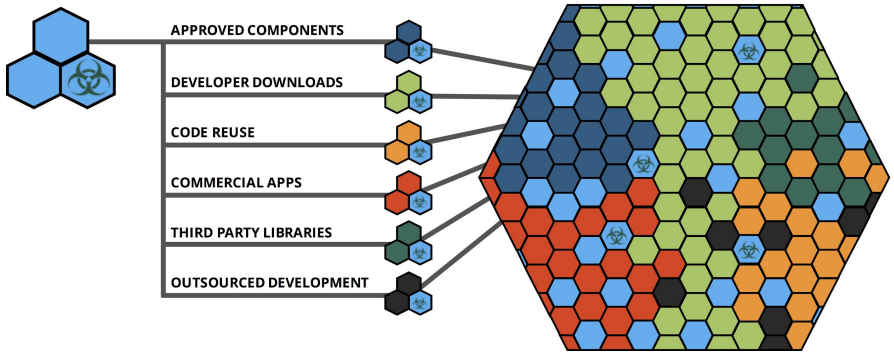
- Resource leaks where resources (e.g., memory, locks, database connections) are not freed on every code path reachable from an allocation
- Code does include conditional checks on every possible error code that can be returned by an API (cannot always be determined statically)
- Using a 'blacklisted' library, service, framework with known security vulnerabilities
- More…

Not same as Bug Trackers like JIRA or Bugzilla, which your teams should also use


Source Code
Model Extraction
Intermediate Representations (IR)
Analysis
Results
Names Databases/Symbol Table
Name Kind Location
copy_item function item. c:25
item_cache variable itemc:10
color parameter pallette.c:23
header.h file chapes.c
Abstract Syntax Tree (AST)
Control Flow graph (CFG)
Call Graph

Example Bug Patterns





Tree: 6d33b... / [redacted] / aws / claudia / **server.js**

[redacted] aws_exercises

1 contributor

21 lines (15 sloc) | 361 Bytes

```
1  var ApiBuilder = require('claudia-api-builder'),
2      api = new ApiBuilder();
3
4  module.exports = api;
5
6  AWS.config.update({
7    "accessKeyId": "AKIA[redacted]",
8    "secretAccessKey": "[redacted]",
9  }
10 );
```

Another Example Bug Pattern: Null Pointers

```
int a, b, c; // some integers
int *pi;     // a pointer to an
integer

a = 5;
pi = &a; // pi points to a
b = *pi; // b is now 5
pi = NULL;
c = *pi; // this is a NULL
pointer dereference
```

```
int a, b, c; // some integers
int *pi;     // a pointer to an
integer

a = 5;
pi = &a; // pi points to a
b = *pi; // b is now 5
pi = abracadabra(a, b);
c = *pi; // could pi be a NULL
pointer?
```

You're dereferencing a null pointer!

Example Unsanitized User Input


```
if (loginSuccessful) {
  logger.severe("User login succeeded
for: " + username);
} else {
  logger.severe("User login failed for: "
+ username);
}
```

Say user has entered username “guest”, the log gets

May 15, 2023 2:19:10 PM

java.util.logging.LogManager$RootLogger log

SEVERE: User login failed for: guest

Say the user entered username “guest

May 15, 2023 2:25:52 PM

java.util.logging.LogManager$RootLogger log

SEVERE: User login succeeded for: administrator“

The log gets

May 15, 2023 2:19:10 PM

java.util.logging.LogManager$RootLogger log

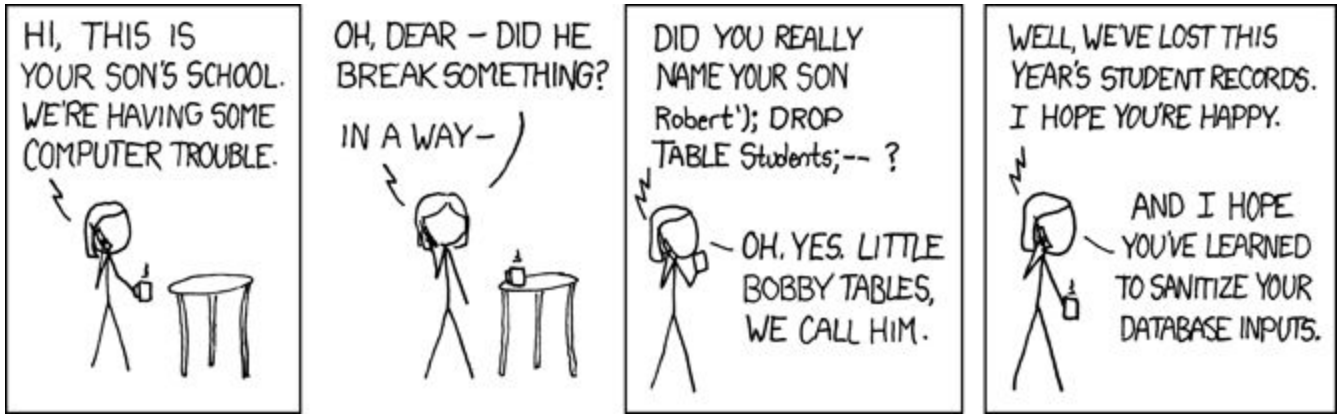
SEVERE: User login failed for: guest

May 15, 2023 2:25:52 PM

java.util.logging.LogManager log

SEVERE: User login succeeded for: administrator

Another Example Unsanitized Input



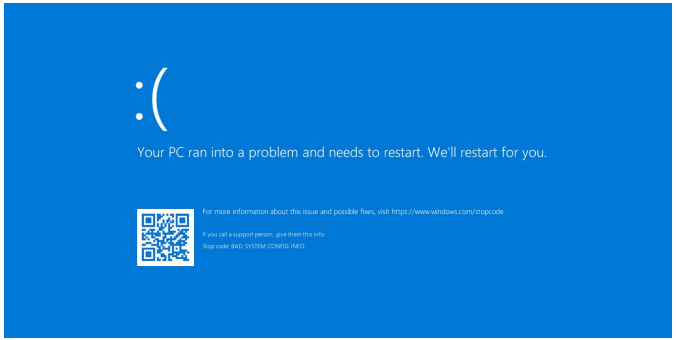
Malicious inputs can come from network, files, devices, even database itself, not just direct user inputs. All inputs should be validated, and non-compliant inputs should be rejected or sanitized

Should your Team Projects Validate Inputs?

Run static analysis bug finder that checks source code for validation/sanitation patterns (and other bug patterns) as well as *test* with invalid inputs

Are All of these Bugs 'Security Vulnerabilities'?

Although using known vulnerable or malicious code, hardwiring secret keys into the code, and processing unsanitized user input are obvious security issues, why do resource leaks and dereferencing null pointers matter from a security perspective?

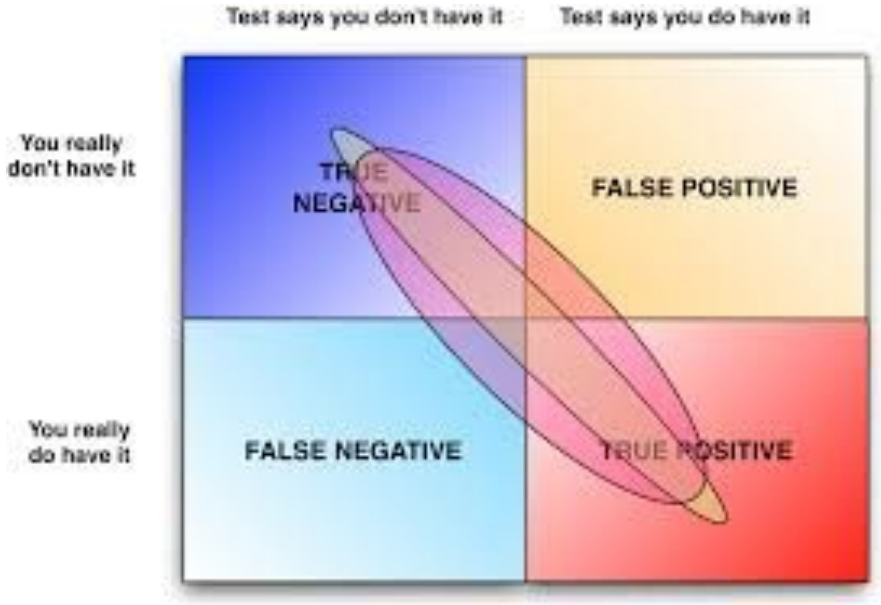


But these bugs might not be exploitable. Why not?

False Positives and False Negatives

Static analysis reports bugs that *might* happen with some inputs because it considers all paths through the program, many of which might not be feasible at runtime - false positives

Dynamic analysis reports bugs that *did* happen with specific inputs under specific circumstances in a specific environment, but exhaustive testing is infeasible so misses many bugs - false negatives



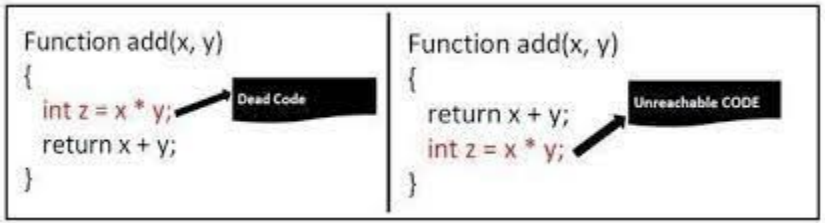
Example Infeasible Path: Unreachable Code

Is this a false positive or a true bug?

```
function foo (int x) {
    if (x<0) { do something buggy }
    else { do something not buggy }
}
```

What if bar is the only function that calls foo?

```
function bar (int y) {
    if (y<0) return;
    foo(y)
}
```



Agenda

1. Looking for Group
2. Finding Bugs by examining the code
3. deferred until a later date: Finding Bugs by executing the code

Upcoming Assignments

- ➔ Team Formation due next Monday, September 18, by 11:59pm
- ➔ Team Preliminary Project Proposal due Monday, September 25, by 11:59pm

Next Week

Anything we didn't finish this week

More on

Reminder: Attendance

We will start taking attendance next Tuesday using zoom's 'usage reports'

These show each attendee's zoom name, join time, leave time, and duration for everyone who attended that zoom session

Make sure your zoom name matches your courseworks name, or you will not get credit for attending. If there is some reason you cannot make them match you need to explain (by posting a private message to the teaching staff, not just me, in edstem)

Make sure you stay for the whole session unless you tell us (by posting a private message to the teaching staff, not just me, in edstem) why you had to arrive late and/or leave early

If you have a bad connection and you manually disconnect and reconnect, or zoom disconnects you and you reconnect, we can put the two (or more) together as long as it's the same zoom name

Ask Me Anything